

Module 52 Geographic Information Systems

Learning goals

- Understand the difference between vector and raster data, and when to use which.
- Learn how to work with vector and raster data in base maps in `R`.
- Learn how to work with vector and raster data in `leaflet` maps.

For some good examples of bad maps – the kind of thing we are trying to help you *avoid* – check out [this Twitter feed](#).

We don't often read scatter plots in life, but we read maps *all the time*. Learning how to work with spatial data and create beautiful maps are essential skills as a data scientist. To get there, you need to become familiar with Geographical Information Systems, or **GIS**.

In a GIS, spatial data are tied together in a database that makes it (1) easy to map the data and (2) easy to access all sorts of information underlying each spatial feature.

GIS matters because most data, it turns out, have an important geographic component. There are nearly *always* important geographic questions you can ask about your data.

Geographic data structures

In your work with spatial data, you are going to encounter **two main types of data**:

Vectors

Vectors are discrete shapes built up by individual points.

There are three common types of vectors:

1. A **point** is a single location (zero-dimensional).
2. A **line** is a set of points connected by straight lines (1-dimensional).
3. A **polygon** is a shape formed by a line whose beginning and end are the same location: it is a line that begins and ends at the same spot and thus encloses an area. Polygons have an inside and an outside.

Vectors are best used to represent discrete objects or areas: the location of an observation, a river, a road, a building, a country's boundary, etc.

Most vectors have two types of data: geographic information, such as coordinates, and tabular data with attributes for the vector. For this reason, vectors are usually saved in **shapefiles**, which typically require a folder of several different files to use. That's why most **shapefiles** are downloaded as zipped folders.

Rasters

Rasters are grids. They have rows and columns, just like a dataframe, and each grid cell represents a single value. Rasters are best used for continuous phenomena – in other words, things that occur *everywhere*: temperature, elevation, humidity, etc.

We interact with rasters all the time: think of smartphone screens and digital photos. Such images are simply rasters of pixels. Each pixel has a value representing color and brightness.

Getting to work

To use R as a GIS, first download a bunch of libraries that are likely to be useful:

```
pkgs <- c('tigris', 'dplyr', 'readr', 'ggplot2', 'leaflet', 'rgdal', 'raster', 'sp', 'rasterVis',  
          'htmltools', 'RColorBrewer', 'leaflet.extras', 'geosphere')  
  
for(pkg in pkgs){  
  if(! pkg %in% installed.packages()){install.packages(pkg)}  
}
```

```
library(dplyr)  
library(readr)  
library(ggplot2)  
library(leaflet)  
library(rgdal)  
library(raster)  
library(sp)  
library(rasterVis)  
library(htmltools)  
library(RColorBrewer)  
library(geosphere)
```

Some learning exercises with Tennessee zip codes

We're going to do some exercises with Tennessee zip codes.

1. What is a zip code?
2. What kind of data format is zip code data likely to be in?
3. Load up some tabular data on zip codes by running the below:

```
tn <- read_csv('https://github.com/databrew/intro-to-data-science/raw/main/data/tnzips.csv')
```

4. Inspect the data. What is the unit of observation.
5. Make a histogram of population.
6. Which zip code is the most populous?
7. Which city has the most zip codes?
8. What is the total population of Tennessee?
9. Make a chart of the 5 most populous cities in Tennessee, showing their population.
10. Load up some *geographic* data on Tennessee zip codes by running the below:

```
library(tigris)
options(tigris_use_cache = TRUE)
temp <- tigris::zctas(starts_with = 370:385)
tnz <- as(temp, 'Spatial')
```

11. Plot the map using the `plot` function
12. Run the below in order to convert the format to a ggplot-friendly dataframe.

```
tng <- fortify(tnz, region = 'GEOID20') %>%
  mutate(zip = id)
```

13. Make a ggplot of the `tng` object.
14. In `tnz`, make a new variable named `percent_water` which shows the percentage of each zip code that is water (in area).
15. Bring data from `tnz` into `tng` by running the below:

```
tng <- left_join(tng, tnz@data, by = c('zip' = 'GEOID20'))
```

16. Modify the above ggplot so that the filled color of each polygon reflects the new variable, `percent_water`.
17. Make a leaflet map with no polygons on it.
18. Add the Tennessee zip codes to the leaflet map.

19. Make the width of the lines thinner and make the background totally transparent.
20. Bring the `pop` column of `tn` into `tnz` using `left_join`.
21. We want to make a map showing population density. We'll start by making a color palette:

```
map_palette <- colorNumeric(palette = brewer.pal(9, "Greens"),  
                             domain=tnz@data$pop,  
                             na.color="#CECECE")
```

22. Make a choropleth of population in leaflet.

```
leaflet() %>%  
  addTiles() %>%  
  addPolygons(data = tnz, fillColor = ~map_palette(pop),  
              fillOpacity = 0.9,  
              stroke=TRUE,  
              color='black',  
              weight=1,  
              label = ~pop)  
Error in eval(f[[2]]), metaData(data), environment(f)): object 'pop' not found
```

23. Make a choropleth of percentage water in leaflet. Make it blue.
24. Create a dataframe of the “centroid” of each zip code by running the below:

```
centroids <- coordinates(tnz)
```

25. Define the location, in longitude, latitude, of Sewanee

```
sewanee <- c(-85.9210899, 35.2031373)
```

26. Use `distsm` from the `geosphere` package to get the distance from Sewanee to each zip code's centroid.

```
km <- distm(x = sewanee, y = centroids)
```

27. Deposit the `distances` object as a column in the `tnz` object

```
tnz@data$km <- km[,1]
```

28. Make a leaflet choropleth of distance to Sewanee.

Working with rasters

To download some raster data, use the function `getData()` from the `raster` package:

```
usa <- getData('alt', country='USA', mask=TRUE)
```

This function is great for getting country-level data. Use `ccodes()` to get the codes for each country:

```
ccodes() %>% head()
```

	NAME	ISO3	ISO2	NAME_ISO	NAME_FAO
1	Afghanistan	AFG	AF	AFGHANISTAN	Afghanistan
2	Akrotiri and Dhekelia	XAD	<NA>	<NA>	<NA>
3	Åland	ALA	AX	ÅLAND ISLANDS	<NA>
4	Albania	ALB	AL	ALBANIA	Albania
5	Algeria	DZA	DZ	ALGERIA	Algeria
6	American Samoa	ASM	AS	AMERICAN SAMOA	American Samoa

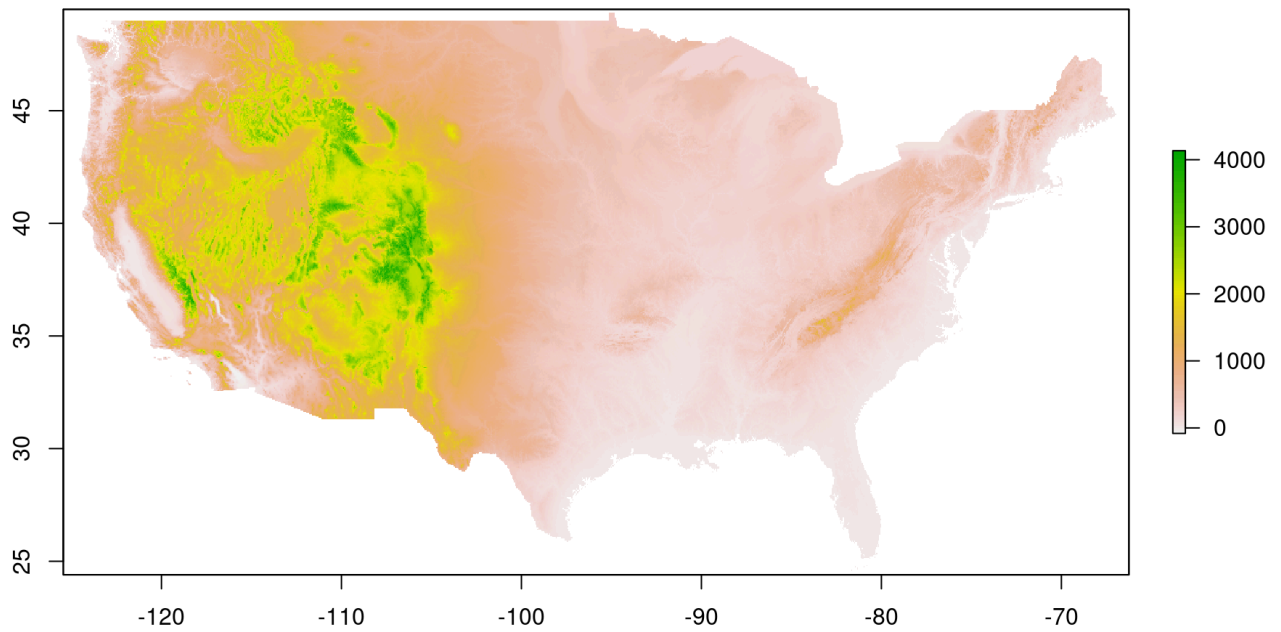
	NAME_LOCAL	SOVEREIGN	UNREGION1	UNREGION2	continent
1	Afghanestan	Afghanistan	Southern Asia	Asia	Asia
2	Akrotiri and Dhekelia	United Kingdom	Western Asia	Asia	Asia
3	Åland	Finland	Northern Europe	Europe	Europe
4	Shqiperia	Albania	Southern Europe	Europe	Europe
5	Al Jaza'ir	Algeria	Northern Africa	Africa	Africa
6	American Samoa	United States	Polynesia	Oceania	Oceania

Since the United States consists of several distant territories (Hawaii, Alaska, Samoa, etc.), the object `usa` is actually a *list* containing an object for each territory:

```
names(usa)
[1] "/home/joebrew/Documents/intro-to-data-science/USA1_msk_alt.grd"
[2] "/home/joebrew/Documents/intro-to-data-science/USA2_msk_alt.grd"
[3] "/home/joebrew/Documents/intro-to-data-science/USA3_msk_alt.grd"
[4] "/home/joebrew/Documents/intro-to-data-science/USA4_msk_alt.grd"
```

Let's plot the continental United States:

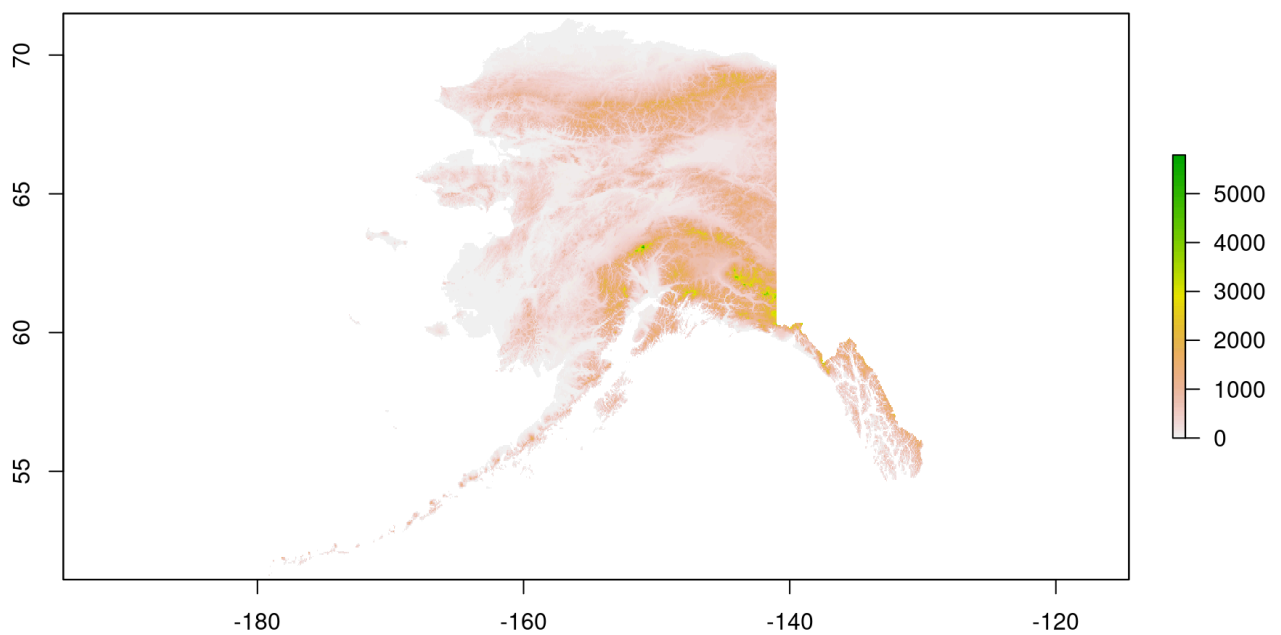
```
plot(usa[[1]])
```



How do you interpret this map? What type of data does this raster object contain?

Now plot a separate territory:

```
plot(usa[[2]])
```



Working with vectors

You can also use the `getData()` function to get vectors, such as the boundaries of U.S. states:

```
states <- getData(name = 'GADM', level = 1, country = 'USA')
```

Check out this `states` object. It's a new type of data structure: a `SpatialPolygonsDataFrame` :

```
head(states)
```

	GID_0	NAME_0	GID_1	NAME_1	VARNAME_1	NL_NAME_1	TYPE_1	ENGTYPE_1
1	USA	United States	USA.1_1	Alabama	AL Ala.	<NA>	State	State
12	USA	United States	USA.2_1	Alaska	AK Alaska	<NA>	State	State
23	USA	United States	USA.3_1	Arizona	AZ Ariz.	<NA>	State	State
34	USA	United States	USA.4_1	Arkansas	AR Ark.	<NA>	State	State
45	USA	United States	USA.5_1	California	CA Calif.	<NA>	State	State
48	USA	United States	USA.6_1	Colorado	CO Colo.	<NA>	State	State

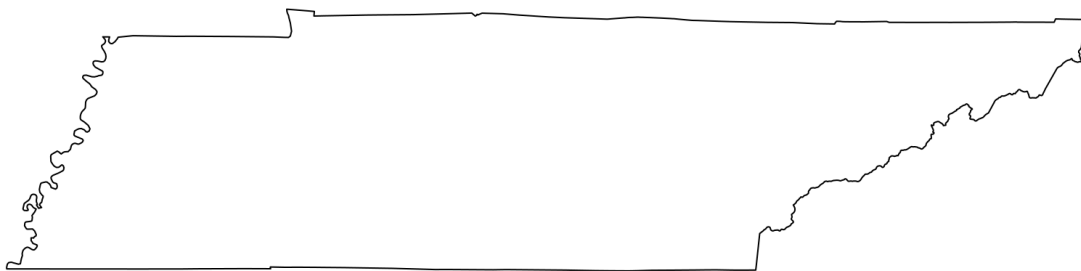
	CC_1	HASC_1
1	<NA>	US.AL
12	<NA>	US.AK
23	<NA>	US.AZ
34	<NA>	US.AR
45	<NA>	US.CA
48	<NA>	US.CO

Subset this object to only the data for Tennessee:

```
tn <- states[states$NAME_1 == 'Tennessee',]
```

Now plot it:

```
plot(tn)
```



Combining rasters & vectors

Now let's try to combine raster data, such as elevation (stored in the object `usa[[1]]`), with vector data, such as state boundaries (stored in the object `tn`).

To do so, let's `crop` the elevation raster to only the data contained within the Tennessee boundary:

```
# First crop to a simple bounding box
```

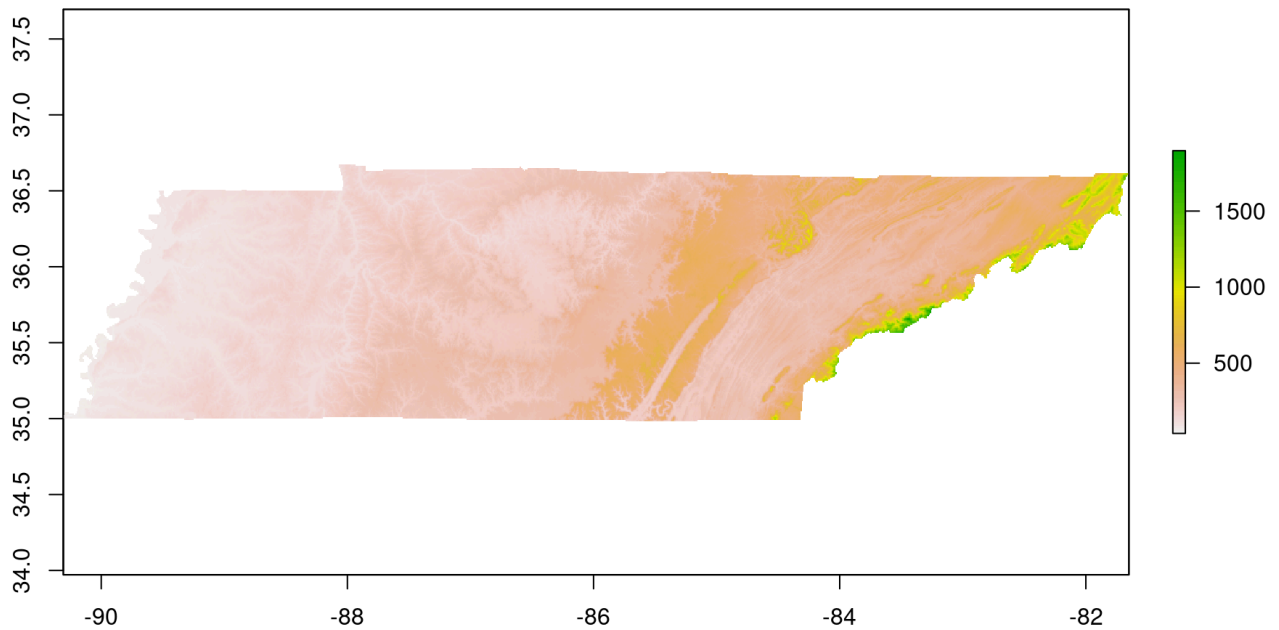
```
tn_elev <- crop(usa[[1]], tn)
```

```
# Now crop to the exact boundary of TN
```

```
tn_elev <- mask(tn_elev, tn)
```

```
# Plot it
```

```
plot(tn_elev)
```

Dealing with projections

Let's load in some data specifically for the area of Sewanee, TN:

```
# Setup a temporary directory for these data
destination_directory <- '/tmp'

# Download the zipped folder of data
destination_file <- file.path(destination_directory, 'sewanee.zip')
download.file('https://raw.githubusercontent.com/databrew/intro-to-data-science/main/data/sewanee.',
              destfile = destination_file)

# Unzip folder
unzip(destination_file, exdir = destination_directory)
```

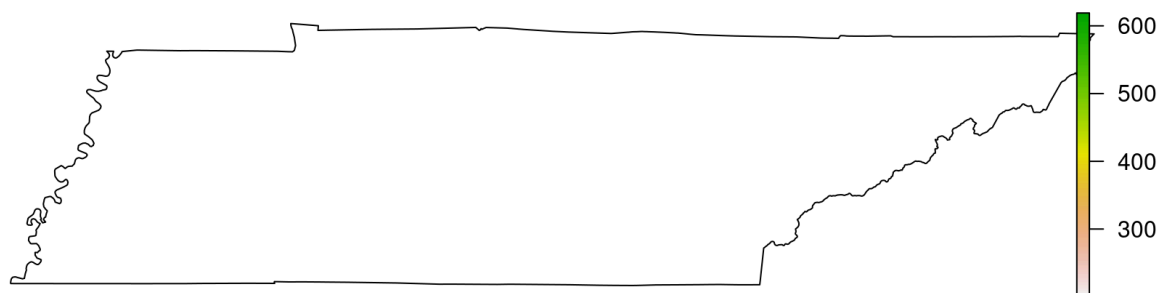
This zipped folder has data files for both rasters and vectors.

Read in the raster data for elevation:

```
elevation <- raster(file.path(destination_directory,
                              'DEM USGS 10m.tif'))
```

Now let's try to map Sewanee's raster data onto the `tn` vector of boundaries:

```
plot(tn)
plot(elevation, add = T)
```



Oh no! That didn't work. Why not? Have a look at coordinates:

```
coordinates(tn) %>% head()
      [,1]      [,2]
38 -86.34243 35.84419
```

```
coordinates(elevation) %>% head()
      x      y
[1,] 587000 3905750
[2,] 587010 3905750
[3,] 587020 3905750
[4,] 587030 3905750
[5,] 587040 3905750
[6,] 587050 3905750
```

It seems like these two objects are using different coordinate systems. Let's check out what *projection* these objects are using:

```
proj4string(elevation)
[1] "+proj=utm +zone=16 +datum=WGS84 +units=m +no_defs"

proj4string(tn)
[1] "+proj=longlat +datum=WGS84 +no_defs"
```

The `tn` coordinates are in classic lat/long, but the `elevation` coordinates appear to be in UTM (*Easting/Northing* coordinates).

Let's convert `elevation`'s coordinate system to that of `tn`:

```
elevation_ll <- projectRaster(elevation,
                             crs = proj4string(tn))
```

Great – now let's try out plot again:

```
plot(tn)
plot(elevation_ll, add = T)
```



Phew – worked this time (even though it still ain't pretty).

Now let's bring in some vectors pertaining to the Sewanee Domain: boundaries, roads, and structures:

```

boundary <- readOGR(destination_directory, 'Boundary2016')
OGR data source with driver: ESRI Shapefile
Source: "/tmp", layer: "Boundary2016"
with 7 features
It has 2 fields

structures <- readOGR(destination_directory, 'Domain_Structures')
OGR data source with driver: ESRI Shapefile
Source: "/tmp", layer: "Domain_Structures"
with 1055 features
It has 113 fields
Integer64 fields read as strings: Total_ft2 ft2_ea_flr

roads <- readOGR(destination_directory, 'Roads')
OGR data source with driver: ESRI Shapefile
Source: "/tmp", layer: "Roads"
with 404 features
It has 18 fields
Integer64 fields read as strings: ID ID2 Column

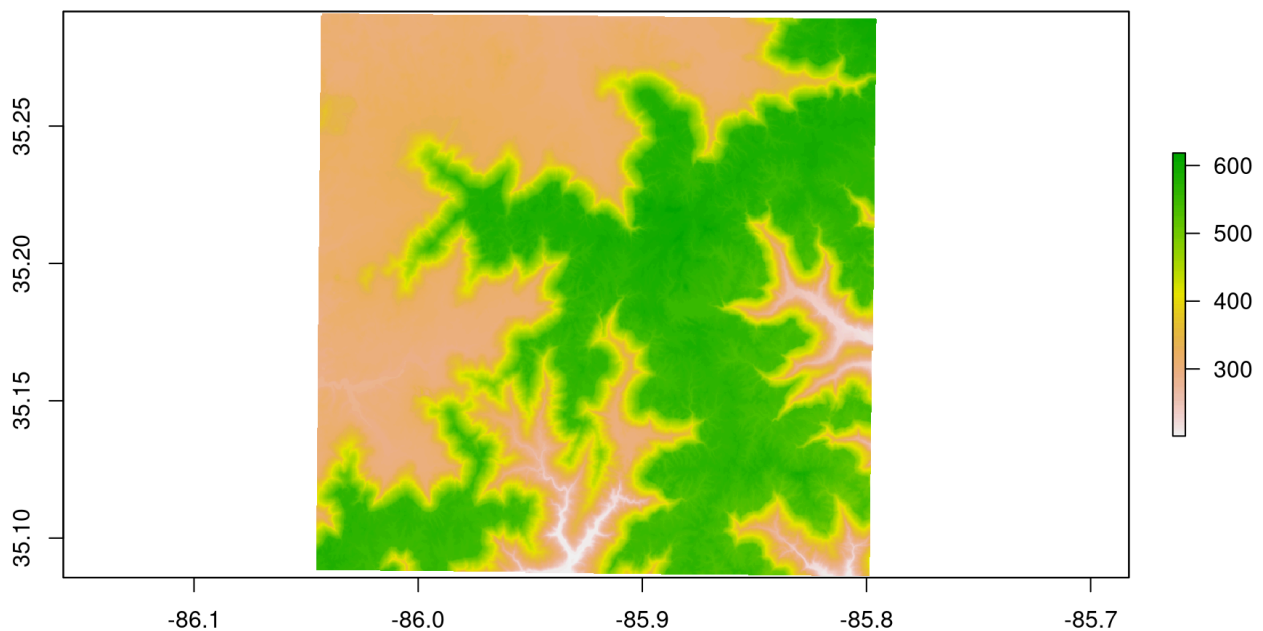
```

This time let's make a zoomed-in plot of Seawanee's land, with elevation and the Domain boundary:

```

plot(elevation_ll)
plot(boundary, add = T)

```



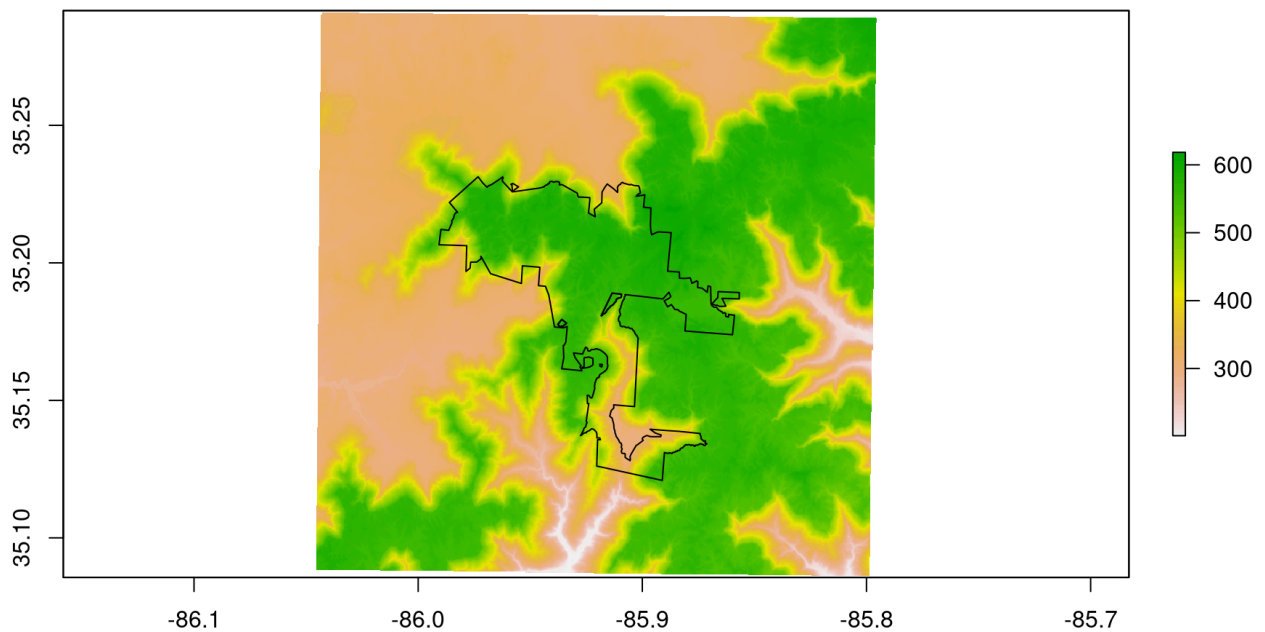
Uh oh. Same problem as before. We need to “reproject” boundary to latitude longitude.

This time, since we are now transforming a vector, we need to use the function `spTransform()` instead of `projectRaster()` .

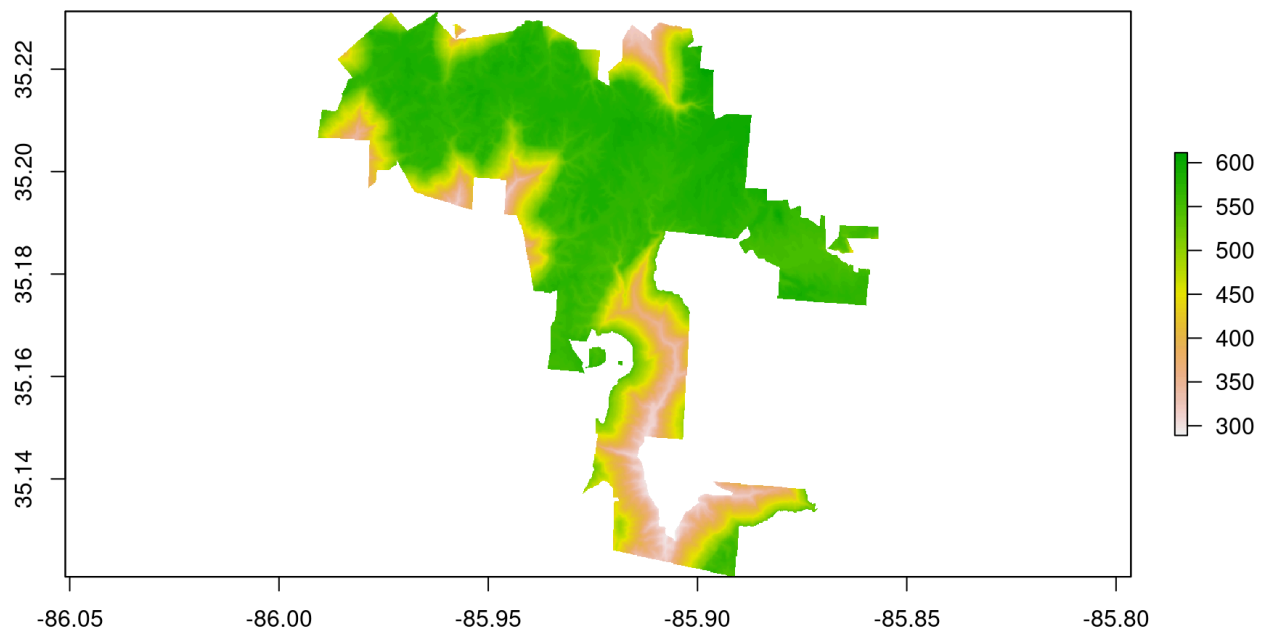
```
boundary_11 <- spTransform(boundary, proj4string(elevation_11))
```

Okay, let's retry our plot:

```
plot(elevation_11)  
plot(boundary_11, add = T)
```

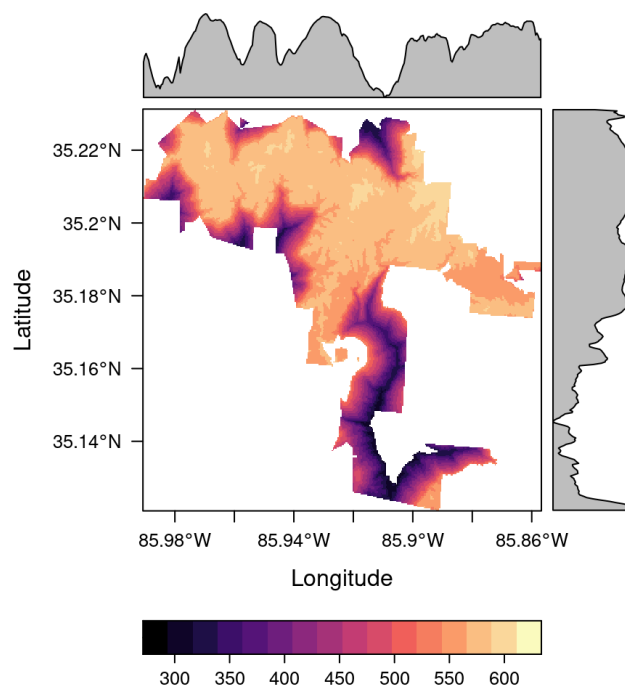


Let's use `crop` and `mask` to get just the elevation for the domain.

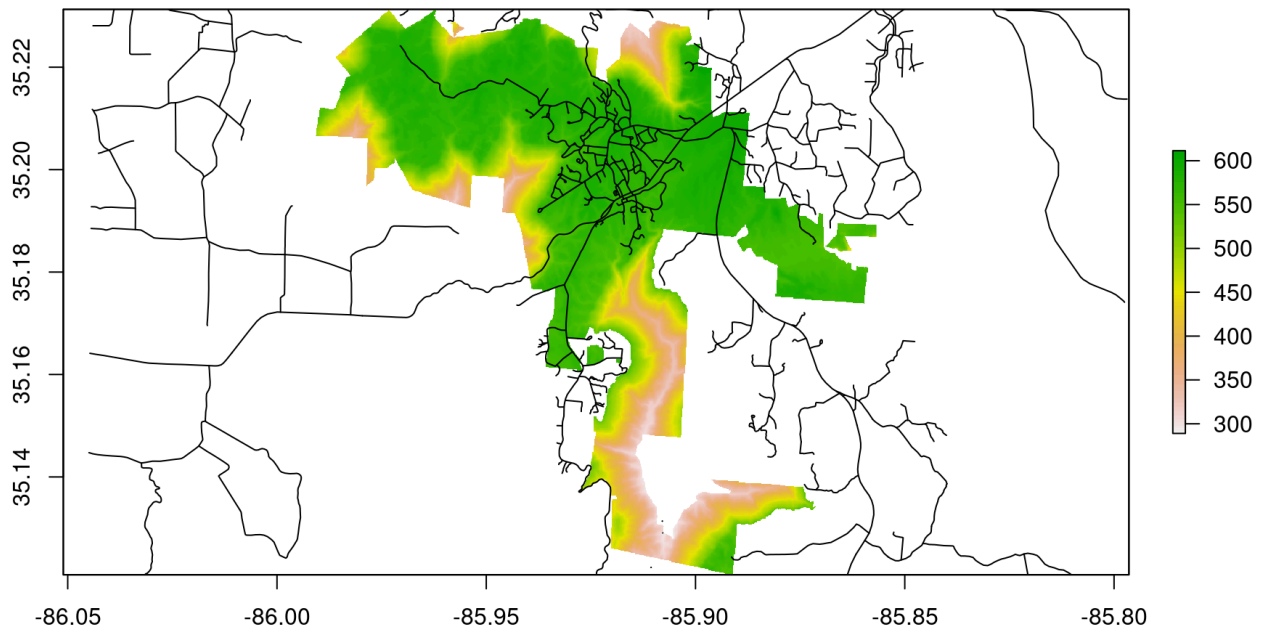


If we wanted more information about this `domain_elev` raster, we could use some special exploration functions for rasters:

```
rasterVis::levelplot(domain_elev)
```



Now let's add roads to the plot. This time, we will preemptively re-project the `roads` data.

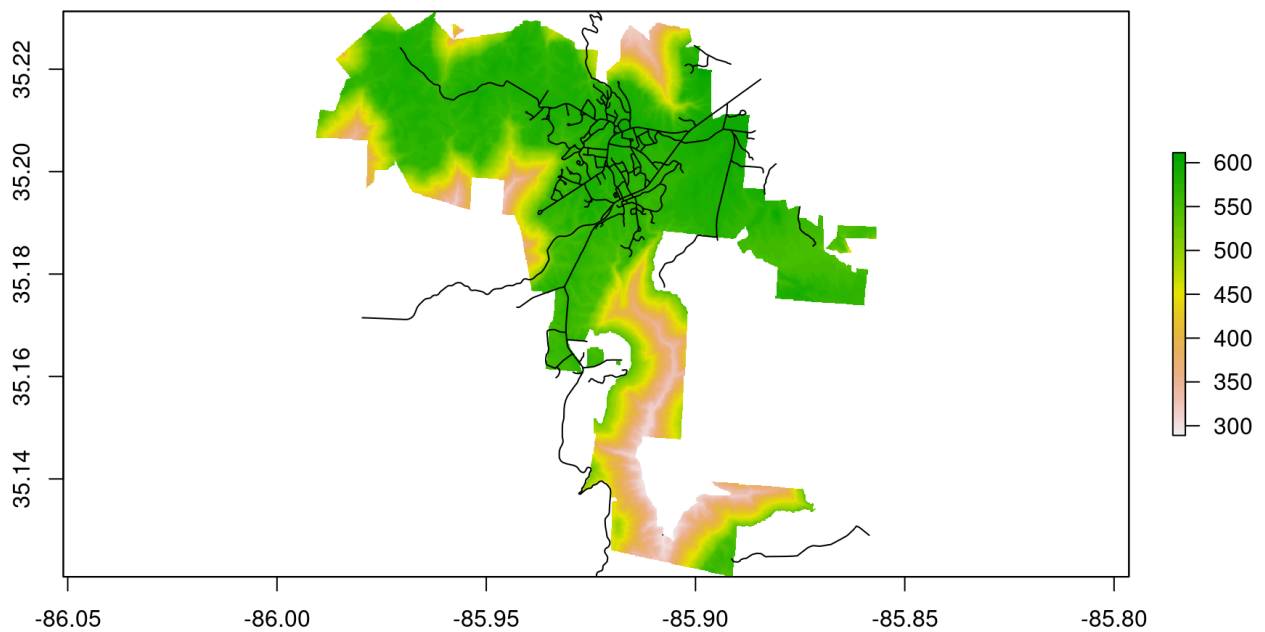


What if we only want to use the roads that cross the Domain boundary? To do so, we will use the `over()` function:

```
o <- over(roads_ll, polygons(boundary_ll))  
roads_ll_trim <- roads_ll[!is.na(o),]
```

Now let's re-do the plot:

```
plot(domain_elev)  
plot(roads_ll_trim, add = TRUE)
```



Combining rasters, vectors, & leaflet

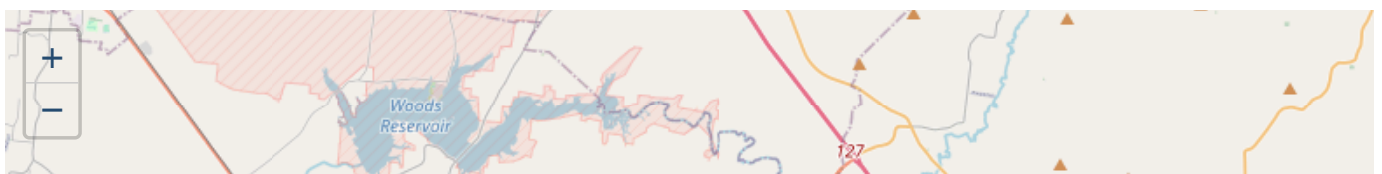
Let's place Sewanee's elevation raster onto a `leaflet` map.

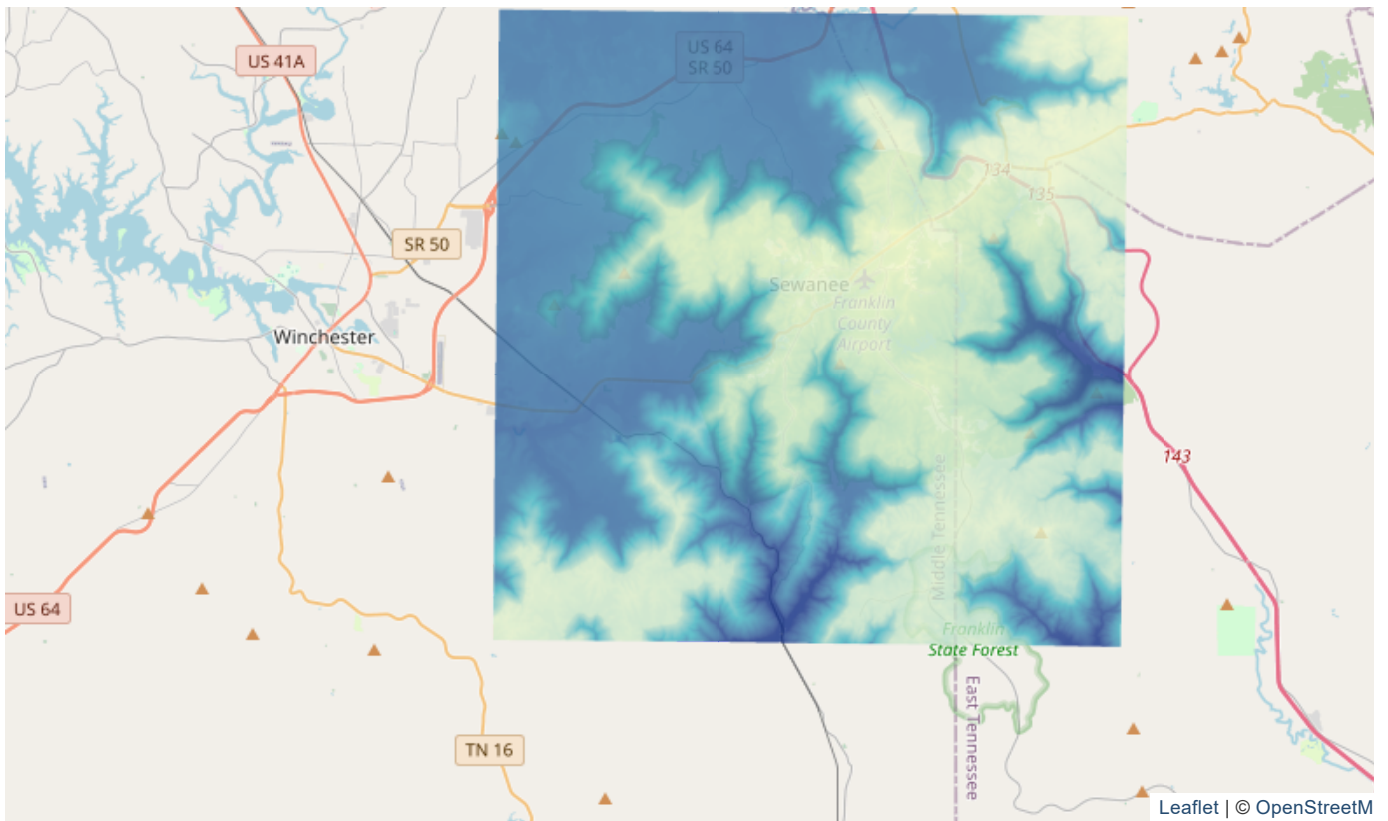
First, let's make a color palette (i.e., a **choropleth**) to represent elevation:

```
pal <- colorNumeric(c("#0C2C84", "#41B6C4", "#FFFFCC"),
  values(elevation_ll),
  na.color = "transparent")
```

We add the raster to the `leaflet` map using the `addRasterImage()` function:

```
leaflet() %>%
  addTiles() %>%
  addRasterImage(elevation_ll,
    colors = pal,
    opacity = 0.8) %>%
  addLegend(pal = pal,
    values = values(elevation_ll),
    title = "Elevation (m)")
```





Exercices

Adding to the Sewanee leaflet map

1. Add structures to your leaflet map of elevation_11 from above. *Hint:* you'll need to use `addPolygons` and you'll need to reproject structures as `structures_11` ...
2. Add popups to your structures.
3. Make your structures a different color and remove the border *Hint:8 you'll need to use the `stroke` argument.
4. Get the elevation of each structure by running:

```
structure_elevation <-
  unlist(lapply(extract(elevation_11, structures_11),
    function(x){
      mean(x, na.rm = TRUE)
    })))
```

4. Use the `structure_elevation` object to add a new column to the `structures_11` object.
5. Make a histogram of the elevation of Sewanee buildings.
6. How low is the lowest building on the domain? How low is it?
7. How high is the highest building on the domain? Which building is it?

A global map of world health

Let's read in a new dataset of polygons, this time the national boundaries of the world's nations:

OGR data source with driver: ESRI Shapefile

Source: "/tmp/world_small", layer: "TM_WORLD_BORDERS_SIMPL-0.3"

with 246 features

It has 11 fields

Integer64 fields read as strings: POP2005

```
# Stage some variables
```

```
file_source <- 'https://raw.githubusercontent.com/databrew/intro-to-data-science/main/data/world_sl
```

```
destination_directory <- '/tmp'
```

```
destination_file <- file.path(destination_directory, 'world_shp.RData')
```

```
# Download the data, if you don't already have it
```

```
if(!'data/world_shp.RData' %in% destination_directory){
```

```
  download.file(file_source,
```

```
                destfile = destination_file)
```

```
}
```

```
load(destination_file)
```

```
# Rename shapefile object
```

```
shp <- world_shp
```



Now let's read in some indicator data of global health:

```
# Read in indicator data
```

```
df <- read_csv('https://raw.githubusercontent.com/databrew/intro-to-data-science/main/data/hefpi.c:
```



8. Check out this data:

```
head(df)
# A tibble: 6 × 8
  ...1 country      ISO3 region_name year indicator_name value unit_of_measure
  <dbl> <chr>      <chr> <chr>      <dbl> <chr>      <dbl> <chr>
1     1 Aruba      ABW  Latin Amer... 2006 BMI, adults    29.5 BMI
2     2 United Ara... ARE  Middle Eas... 2003 BMI, adults    26.2 BMI
3     3 Armenia     ARM  Europe & C... 2016 BMI, adults    25.8 BMI
4     4 American S... ASM  East Asia ... 2004 BMI, adults    34.9 BMI
5     5 Australia   AUS  East Asia ... 2012 BMI, adults    26.8 BMI
6     6 Austria     AUT  Europe & C... 2006 BMI, adults    25.4 BMI

nrow(df)
[1] 6224
```

9. Let's focus in on the rate of inpatient care use among adults:

```
pd <- df %>% filter(indicator_name == 'Inpatient care use, adults')
```

10. Now let's join this tabular data to the shapefile data according to country:

```
shp@data <- left_join(shp@data, pd)
```

11. OK, let's get a map started. First, let's establish our base map:

```
leaf <- leaflet(shp) %>%
  addProviderTiles(provider = providers$Esri.WorldShadedRelief)
```

12. Next, let's create a color scale for the value of interest:

```
map_palette <- colorNumeric(palette = brewer.pal(9, "Greens"),
  domain=shp@data$value,
  na.color="#CECECE")
```

13. And use this palette to color-code each country:

```
leaf <- leaf %>% addPolygons(fillColor = ~map_palette(value),
  fillOpacity = 0.9,
  stroke=TRUE,
  color='black',
  weight=1,
  label = ~round(value,2))
```

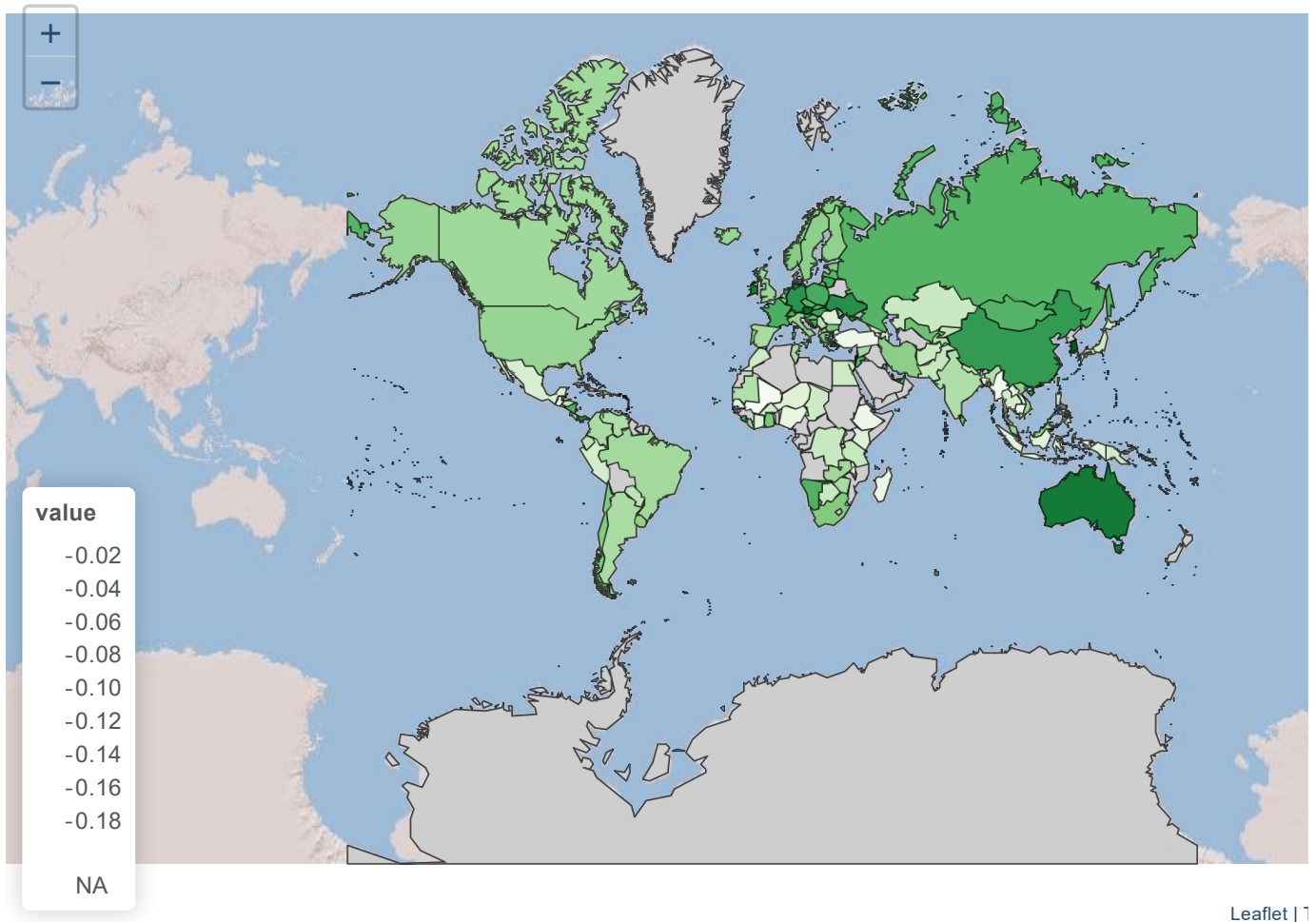
14. Now add a legend:

```
leaf <- leaf %>% addLegend(pal=map_palette,  
                           values=~value,  
                           opacity=0.9,  
                           position = "bottomleft",  
                           na.label = "NA" )
```

15. Let's stage some fancy text:

```
map_text <- paste(  
  "Indicator: ", shp@data$indicator_name,"<br>",  
  "Economy: ", as.character(shp@data$country),"<br/>",  
  'Value: ', round(shp@data$value, digits = 2), "<br/>",  
  "Year: ", as.character(shp@data$year),"<br/>",sep="") %>%  
  lapply(htmltools::HTML)
```

```
leaf <- leaf %>%  
  addPolygons(  
    color = 'black',  
    fillColor = ~map_palette(value),  
    stroke=TRUE,  
    fillOpacity = 0.9,  
    weight=1,  
    label = map_text,  
    highlightOptions = highlightOptions(  
      weight = 1,  
      fillColor = 'white',  
      fillOpacity = 1,  
      color = "white",  
      opacity = 1.0,  
      bringToFront = TRUE,  
      sendToBack = TRUE  
    ),  
    labelOptions = labelOptions(  
      noHide = FALSE,  
      style = list("font-weight" = "normal", padding = "3px 8px"),  
      textsize = "13px",  
      direction = "auto"  
    )  
  )  
  
leaf
```



16. Now make a choropleth map of BMI for men, where the darker the shade of red, the higher the BMI for each country.

17. Remove the borders from the map

18. Add a legend on the top right of the map

19. Make the NA color blue

20. Make the hover label a combination of the country and BMI value

21. Make the title of the legend "BMI"

22. Create a function that takes an indicator name as an input and creates a map.